Top 100 Advanced Python Interview Questions with Answers and Examples

1. Decorators

Q1: Write a decorator that caches return values of a function (LRU cache) without using `functools.lru_cache`.

A: Use a dictionary with a queue to track usage. Here's a simple LRU cache decorator:

```
from collections import OrderedDict
```

```
def lru_cache(maxsize=128):
    def decorator(func):
        cache = OrderedDict()
        def wrapper(*args, **kwargs
            key = (args, tuple(kwar
            if key in cache:
                cache.move_to_end(k
                return cache[key]
            result = func(*args, **
            cache[key] = result
            if len(cache) > maxsize
                cache.popitem(last=
            return result
        return wrapper
    return decorator

@lru_cache(maxsize=2)
def slow_square(x):
    return x * x

print(slow_square(3))  # computed
print(slow_square(3))  # cached
```

Q2: How can you preserve metadata of the original

function when using decorators?

A: Use `functools.wraps` to copy `__name__`, `__doc__`, etc.

```
from functools import wraps

def debug(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__nam
        return func(*args, **kwargs
    return wrapper

@debug
def add(a, b):
    """Add two numbers."""
    return a + b

print(add.__name__)   # 'add' (not
print(add.__doc__)    # 'Add two nu
```

Q3: Implement a decorator that retries a function on

exception with exponential backoff.

A:

```
import time
from functools import wraps

def retry(max_retries=3, delay=1, b
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs
            retries, current_delay
            while retries < max_ret
                try:
                    return func(*ar
                except Exception as
                    retries += 1
                    if retries == m
                        raise
                    time.sleep(curr
                    current_delay *
            return None
        return wrapper
    return decorator

@retry(max_retries=3, delay=0.5)
```

```
def unstable_network_call():
    import random
    if random.random() < 0.7:
        raise ConnectionError("Fail
    return "Success"
```

Q4: Create a decorator that logs method execution time and arguments.

A:

```
import time
from functools import wraps

def log_execution(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwar
        end = time.perf_counter()
        print(f"{func.__name__}({ar
        return result
    return wrapper
```

```
@log_execution
def compute_fib(n):
    if n < 2:
        return n
    return compute_fib(n-1) + compu

compute_fib(10)
```

Q5: Explain decorators with arguments and nested decorators.

A: Decorators with arguments require an extra outer layer. Example with prefix:

```
def prefix_logger(prefix):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs
            print(f"{prefix} - call
            return func(*args, **kw
        return wrapper
    return decorator

@prefix_logger("DEBUG")
```

```
def test():
    pass

# Multiple decorators execute botto
```

Q6: Write a class-based decorator that maintains state.

A:

```
class CountCalls:
    def __init__(self, func):
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwa
        self.count += 1
        print(f"Call {self.count} o
        return self.func(*args, **k

@CountCalls
def say_hello():
    print("Hello")
```

```
say_hello()
say_hello()  # count increments
```

Q7: How to decorate a class (i.e., class decorator)?

A: Class decorators modify or enhance class creation.

```
def add_repr(cls):
    def __repr__(self):
        return f"{cls.__name__}({se
    cls.__repr__ = __repr__
    return cls

@add_repr
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
print(p)  # Point({'x': 1, 'y': 2})
```

Q8: Implement a decorator that validates argument types using type hints.

A:

```
from functools import wraps
from inspect import signature

def type_check(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        sig = signature(func)
        bound = sig.bind(*args, **k
        bound.apply_defaults()
        for name, value in bound.ar
            param = sig.parameters[
            expected = param.annota
            if expected is not para
                raise TypeError(f"{
        return func(*args, **kwargs
    return wrapper

@type_check
def multiply(a: int, b: int) -> int
    return a * b
```

```
multiply(2, 3)   # OK
# multiply(2, "3")  # TypeError
```

Q9: Create a decorator that caches property values (cached property).

A:

```
class cached_property:
    def __init__(self, func):
        self.func = func
        self.name = func.__name__

    def __get__(self, instance, own
        if instance is None:
            return self
        value = self.func(instance)
        instance.__dict__[self.name
        return value

class Circle:
    def __init__(self, radius):
        self.radius = radius
```

```
    @cached_property
    def area(self):
        print("Computing area...")
        return 3.14159 * self.radiu

c = Circle(5)
print(c.area)  # computes
print(c.area)  # cached
```

Q10: How to write a decorator that optionally accepts arguments (flexible)?

A: Detect if the decorator is called with or without parentheses.

```
from functools import wraps

def optional_decorator(func=None, *
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs
            print(prefix + f.__name
```

```
            return f(*args, **kwarg
        return wrapper
    if func is None:
        return decorator
    return decorator(func)

@optional_decorator
def hello():
    print("World")

@optional_decorator(prefix="!! ")
def goodbye():
    print("Bye")

hello()   # >>> hello \n World
goodbye() # !! goodbye \n Bye
```

2. Generators and Iterators

Q11: Explain the difference between `yield` and `yield from`. Provide an example.

A: `yield` returns a value and pauses; `yield from` delegates to a subgenerator (PEP 380).

```
def gen_with_yield():
    for i in range(3):
        yield i

def gen_with_yield_from():
    yield from range(3)   # cleaner

# Nested delegation
def outer():
    yield "start"
    yield from inner()
    yield "end"
```

```
def inner():
    yield 1
    yield 2

print(list(outer()))  # ['start', 1
```

Q12: Implement a generator that reads a large file line by line lazily.

A:

```
def read_large_file(file_path):
    with open(file_path, 'r') as f:
        for line in f:
            yield line.rstrip('\n')

# Memory efficient: only one line i
for line in read_large_file("big.tx
    process(line)
```

Q13: Write a generator that produces an infinite

Fibonacci sequence.

A:

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
first_10 = [next(fib) for _ in rang
print(first_10)  # [0, 1, 1, 2, 3,
```

Q14: How to send values to a generator using `send()`?

A: Generators can receive data; `send()` resumes and passes a value into the generator.

```
def accumulator():
    total = 0
    while True:
```

```
        value = yield total
        if value is None:
            break
        total += value

acc = accumulator()
next(acc)           # initialize, yi
print(acc.send(5)) # total=5, yield
print(acc.send(3)) # total=8, yield
acc.close()
```

Q15: Implement a coroutine using `yield` for cooperative multitasking (producer-consumer).

A:

```
def consumer():
    while True:
        item = yield
        print(f"Consumed {item}")

def producer(consumer, items):
    next(consumer)  # prime the cor
```

```
    for item in items:
        print(f"Produced {item}")
        consumer.send(item)

c = consumer()
producer(c, [1,2,3])
```

Q16: What is the purpose of `itertools.tee()`? Provide example.

A: `tee` creates multiple independent iterators from a single iterable.

```
from itertools import tee

original = [1,2,3,4]
it1, it2 = tee(original, 2)
print(list(it1))  # [1,2,3,4]
print(list(it2))  # [1,2,3,4] - inc
```

Q17: Explain the concept of generator delegation

using `yield from` with `send()` and `throw()`.

A: `yield from` forwards `send()`, `throw()`, and `close()` to the subgenerator.

```
def subgen():
    while True:
        x = yield
        print(f"subgen got {x}")

def delegator():
    yield from subgen()

d = delegator()
next(d)          # prime
d.send(10)       # subgen got 10
d.throw(ValueError)  # propagates
```

Q18: Write a generator that yields running average of a stream of numbers.

A:

```
def running_average():
    total = 0
    count = 0
    while True:
        value = yield
        if value is None:
            break
        total += value
        count += 1
        yield total / count

ra = running_average()
next(ra)
print(ra.send(10))  # 10.0
print(ra.send(20))  # 15.0
print(ra.send(30))  # 20.0
```

Q19: How to convert a generator into a list without consuming it?

A: Not directly possible; you can materialize it and then recreate using `tee`.

```
from itertools import tee

def gen():
    yield from range(5)

g = gen()
g, g_copy = tee(g)
list_copy = list(g_copy)   # consum
print(list_copy)           # [0,1,2
# original g is still intact (but a
```

Q20: Implement a generator that yields permutations of a list without using `itertools`.

A:

```
def permutations(items):
    if len(items) <= 1:
        yield items
    else:
        for i, item in enumerate(it
            rest = items[:i] + item
```

```
            for perm in permutation
                yield [item] + perm

for p in permutations([1,2,3]):
    print(p)
```

3. Context Managers

Q21: Write a context manager using a class to manage a file that logs errors.

A:

```
class ManagedFile:
    def __init__(self, filename, mo
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.filer
        return self.file

    def __exit__(self, exc_type, ex
        if self.file:
            self.file.close()
        if exc_type is not None:
            print(f"Error: {exc_val
```

```
        return False  # propagate e

with ManagedFile("test.txt", "w") a
    f.write("Hello")
```

Q22: Implement a context manager using `contextlib.contextmanager` decorator.

A:

```
from contextlib import contextmanag

@contextmanager
def managed_file(filename, mode='r'
    f = open(filename, mode)
    try:
        yield f
    finally:
        f.close()

with managed_file("test.txt", "w")
    f.write("Using decorator")
```

Q23: Create a context manager that measures execution time.

A:

```
import time
from contextlib import contextmanag

@contextmanager
def timer(name="Block"):
    start = time.perf_counter()
    try:
        yield
    finally:
        elapsed = time.perf_counter
        print(f"{name} took {elapse

with timer("Sleep"):
    time.sleep(0.5)
```

Q24: How to write an asynchronous context

manager (for `async with`)?

A: Define `__aenter__` and `__aexit__`.

```
import asyncio

class AsyncResource:
    async def __aenter__(self):
        print("Acquiring resource")
        await asyncio.sleep(0.1)
        return self

    async def __aexit__(self, exc_t
        print("Releasing resource")
        await asyncio.sleep(0.1)

async def main():
    async with AsyncResource() as r
        print("Inside block")

asyncio.run(main())
```

Q25: Explain `contextlib.ExitStack` and

provide a use case.

A: `ExitStack` dynamically manages multiple context managers, useful when number unknown.

```
from contextlib import ExitStack

files = ['a.txt', 'b.txt', 'c.txt']
with ExitStack() as stack:
    file_handles = [stack.enter_cor
    for fh in file_handles:
        fh.write("data")
# all files closed automatically
```

4. Metaclasses

Q26: What is a metaclass? Write one that automatically adds a `created_at` attribute to all instances.

A: Metaclasses define how classes behave; they are classes of classes.

```
import datetime

class AutoTimestampMeta(type):
    def __call__(cls, *args, **kwar
        instance = super().__call__
        instance.created_at = datet
        return instance

class MyClass(metaclass=AutoTimesta
    pass
```

```
obj = MyClass()
print(obj.created_at)  # current ti
```

Q27: Implement a metaclass that enforces that class methods have docstrings.

A:

```
class DocstringMeta(type):
    def __new__(cls, name, bases, c
        for attr_name, attr_value i
            if callable(attr_value)
                if not attr_value._
                    raise TypeError
        return super().__new__(cls,

class Good(metaclass=DocstringMeta)
    def foo(self):
        """Has docstring"""
        pass

# class Bad(metaclass=DocstringMeta
```

```
#     def bar(self):
#         pass   # Raises TypeError
```

Q28: Explain the difference between `__new__` and `__init__` in metaclasses. {#q28-explain-the-difference-between-new-and-init-in-metaclasses }

A: `__new__` creates the class object; `__init__` initializes it after creation.

```
class Meta(type):
    def __new__(meta, name, bases,
        print(f"Creating class {nam
        dct['added'] = 42
        return super().__new__(meta

    def __init__(cls, name, bases,
        print(f"Initializing class
        super().__init__(name, base

class X(metaclass=Meta):
```

```
    pass

print(X.added)  # 42
```

Q29: How to use a metaclass to implement a singleton pattern?

A:

```
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *args, **kwar
        if cls not in cls._instance
            cls._instances[cls] = s
        return cls._instances[cls]

class Database(metaclass=SingletonM
    def __init__(self):
        print("Init Database")

db1 = Database()
db2 = Database()
print(db1 is db2)  # True
```

Q30: Implement a metaclass that registers all subclasses automatically.

A:

```
class RegistryMeta(type):
    registry = {}

    def __new__(meta, name, bases,
        cls = super().__new__(meta,
        if name not in meta.registr
            meta.registry[name] = c
        return cls

class Base(metaclass=RegistryMeta):
    pass

class A(Base): pass
class B(Base): pass

print(RegistryMeta.registry)  # {'B
```

5. Descriptors

Q31: Explain the descriptor protocol (`__get__` , `__set__` , `__delete__`). Implement a `PositiveNumber` descriptor. {#q31-explain-the-descriptor-protocol-get-set-delete-implement-a-positivenumber-descriptor }

A: Descriptors control attribute access.

```
class PositiveNumber:
    def __set_name__(self, owner, r
        self.name = name

    def __get__(self, instance, owr
        if instance is None:
            return self
```

```
        return instance.__dict__.ge

    def __set__(self, instance, val
        if value <= 0:
            raise ValueError(f"{sel
        instance.__dict__[self.name

    def __delete__(self, instance):
        del instance.__dict__[self.

class Order:
    quantity = PositiveNumber()
    price = PositiveNumber()

    def __init__(self, quantity, pr
        self.quantity = quantity
        self.price = price

o = Order(10, 5.0)
# o.quantity = -5  # ValueError
```

Q32: What is the difference between data descriptor and non-data descriptor?

A: Data descriptor implements `__set__` or `__delete__`; non-data only `__get__`. Data descriptors override instance dictionary.

```
class DataDesc:
    def __get__(self, inst, owner):
    def __set__(self, inst, value):

class NonDataDesc:
    def __get__(self, inst, owner):

class C:
    d = DataDesc()
    nd = NonDataDesc()

c = C()
c.d = 10    # Data set (descriptor
c.nd = 20   # Sets in instance dict
```

Q33: Implement a lazy property using a non-data descriptor.

A:

```
class LazyProperty:
    def __init__(self, func):
        self.func = func
        self.name = func.__name__

    def __get__(self, instance, own
        if instance is None:
            return self
        value = self.func(instance)
        setattr(instance, self.name
        return value

class Data:
    @LazyProperty
    def heavy(self):
        print("Computing heavy...")
        return sum(range(1000000))

d = Data()
print(d.heavy)  # computes
print(d.heavy)  # cached, no recomp
```

Q34: Write a descriptor that validates type and

value range.

A:

```
class TypedProperty:
    def __init__(self, typ, min_val
        self.typ = typ
        self.min_val = min_val
        self.max_val = max_val

    def __set_name__(self, owner, n
        self.name = name

    def __get__(self, instance, own
        if instance is None:
            return self
        return instance.__dict__.ge

    def __set__(self, instance, val
        if not isinstance(value, se
            raise TypeError(f"{self
        if self.min_val is not None
            raise ValueError(f"{sel
        if self.max_val is not None
            raise ValueError(f"{sel
        instance.__dict__[self.name
```

```
class Person:
    age = TypedProperty(int, 0, 150

p = Person()
p.age = 30
```

Q35: Explain how `@property` works under the hood.

A: `property` is a data descriptor that calls getter/setter functions.

```
class Property:
    def __init__(self, fget=None, f
        self.fget = fget
        self.fset = fset
        self.fdel = fdel

    def __get__(self, instance, owr
        if instance is None:
            return self
        if self.fget is None:
            raise AttributeError("u
        return self.fget(instance)
```

```
    def __set__(self, instance, val
        if self.fset is None:
            raise AttributeError("c
        self.fset(instance, value)

    def setter(self, fset):
        return type(self)(self.fget
```

6. Concurrency: Threading, Multiprocessing, Asyncio

Q36: Implement a thread-safe singleton using `threading.Lock`.

A:

```
import threading

class Singleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is
```

```
                    cls._instance =
        return cls._instance
```

Q37: Write a producer-consumer using `queue.Queue` with multiple threads.

A:

```
import threading
import queue
import time
import random

def producer(q, id):
    for i in range(5):
        item = f"Producer-{id}:{i}"
        q.put(item)
        print(f"Produced {item}")
        time.sleep(random.random())

def consumer(q):
    while True:
        item = q.get()
```

```
            if item is None:
                break
            print(f"Consumed {item}")
            q.task_done()

q = queue.Queue()
threads = []
for i in range(2):
    t = threading.Thread(target=pro
    t.start()
    threads.append(t)
cons = threading.Thread(target=cons
cons.start()
for t in threads:
    t.join()
q.put(None)  # signal consumer to s
cons.join()
```

Q38: Explain the Global Interpreter Lock (GIL) and how to bypass it.

A: GIL allows only one thread to execute Python bytecode at a time. Bypass using multiprocessing or C extensions.

```
# Use multiprocessing for CPU-bound
from multiprocessing import Pool

def cpu_intensive(n):
    return sum(i*i for i in range(n

with Pool(4) as p:
    results = p.map(cpu_intensive,
```

Q39: What is `asyncio` and how does it differ from threading? Write an async HTTP fetcher.

A: Asyncio uses single-threaded event loop with coroutines for I/O-bound tasks.

```
import asyncio
import aiohttp

async def fetch(session, url):
    async with session.get(url) as
        return await response.text(
```

```
async def main():
    urls = ['http://example.com', '
    async with aiohttp.ClientSessio
        tasks = [fetch(session, url
        results = await asyncio.gat
        print(len(results))

asyncio.run(main())
```

Q40: Implement a custom `ThreadPool` using `queue.Queue` and threading.

A:

```
import threading
import queue

class ThreadPool:
    def __init__(self, num_threads)
        self.tasks = queue.Queue()
        self.workers = []
        for _ in range(num_threads)
            t = threading.Thread(ta
```

```
            t.daemon = True
            t.start()
            self.workers.append(t)

    def _worker(self):
        while True:
            func, args, kwargs = se
            try:
                func(*args, **kwarg
            except Exception as e:
                print(e)
            finally:
                self.tasks.task_dor

    def submit(self, func, *args, *
        self.tasks.put((func, args,

    def join(self):
        self.tasks.join()
```

Q41: Write a coroutine that times out after `n` seconds using `asyncio.wait_for`.

A:

```
import asyncio

async def long_running():
    await asyncio.sleep(5)
    return "Done"

async def main():
    try:
        result = await asyncio.wait
    except asyncio.TimeoutError:
        print("Timeout!")

asyncio.run(main())
```

Q42: How to share data between processes safely? Use `multiprocessing.Value` and `Array`.

A:

```
from multiprocessing import Process
```

```
def worker(val, arr):
    val.value += 1
    for i in range(len(arr)):
        arr[i] *= 2

shared_val = Value('i', 0)
shared_arr = Array('i', [1,2,3])
p = Process(target=worker, args=(sh
p.start()
p.join()
print(shared_val.value, list(shared
```

Q43: Implement a lock-free queue using `collections.deque` with threading (not truly lock-free but atomic append/pop?).

A: `deque` append/pop are thread-safe due to GIL, but usually need locks for correctness.

```
from collections import deque
import threading

class SimpleQueue:
    def __init__(self):
        self._queue = deque()
        self._lock = threading.Lock

    def put(self, item):
        with self._lock:
            self._queue.append(item

    def get(self):
        with self._lock:
            return self._queue.popl
```

Q44: Write an async generator that yields numbers every second.

A:

```
import asyncio

async def async_countdown(n):
```

```
    for i in range(n, 0, -1):
        yield i
        await asyncio.sleep(1)

async def main():
    async for num in async_countdov
        print(num)

asyncio.run(main())
```

Q45: Explain `concurrent.futures.ThreadPoolExecutor` vs `ProcessPoolExecutor`.

A: ThreadPool for I/O-bound, ProcessPool for CPU-bound.

```
from concurrent.futures import Thre

def io_bound():
    import time; time.sleep(1)

def cpu_bound():
    return sum(i*i for i in range(1
```

```
with ThreadPoolExecutor(max_workers
    executor.map(io_bound, range(10

with ProcessPoolExecutor(max_worker
    results = executor.map(cpu_bour
```

Q46: Implement a barrier using `threading.Barrier`.

A:

```
import threading
import time

def worker(barrier, id):
    print(f"Worker {id} waiting at
    barrier.wait()
    print(f"Worker {id} passed barr

barrier = threading.Barrier(3)
threads = [threading.Thread(target=
for t in threads: t.start()
for t in threads: t.join()
```

Q47: What is `asyncio.Queue`? Example of producer-consumer with asyncio.

A:

```
import asyncio

async def producer(queue):
    for i in range(5):
        await queue.put(i)
        print(f"Produced {i}")
        await asyncio.sleep(0.5)

async def consumer(queue):
    while True:
        item = await queue.get()
        if item is None:
            break
        print(f"Consumed {item}")
        queue.task_done()
        await asyncio.sleep(0.2)

async def main():
    q = asyncio.Queue()
```

```
    prod = asyncio.create_task(proc
    cons = asyncio.create_task(cons
    await prod
    await q.put(None)  # sentinel
    await cons

asyncio.run(main())
```

Q48: Write a thread-safe counter using `threading.Lock`.

A:

```
class ThreadSafeCounter:
    def __init__(self):
        self._value = 0
        self._lock = threading.Lock

    def increment(self):
        with self._lock:
            self._value += 1

    def decrement(self):
        with self._lock:
```

```
            self._value -= 1

    @property
    def value(self):
        with self._lock:
            return self._value
```

Q49: How to cancel a running asyncio task?

A:

```
import asyncio

async def long_task():
    try:
        while True:
            await asyncio.sleep(1)
            print("Working...")
    except asyncio.CancelledError:
        print("Task cancelled")
        raise

async def main():
    task = asyncio.create_task(long
    await asyncio.sleep(3)
```

```
    task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        print("Main: task cancelled

asyncio.run(main())
```

Q50: Implement a semaphore with `threading.Semaphore` to limit concurrent connections.

A:

```
import threading
import time

class ConnectionPool:
    def __init__(self, limit):
        self.semaphore = threading.

    def connect(self, name):
        with self.semaphore:
```

```
            print(f"{name} acquired
            time.sleep(1)
            print(f"{name} released

pool = ConnectionPool(2)
threads = [threading.Thread(target=
for t in threads: t.start()
for t in threads: t.join()
```

7. Memory Management, Garbage Collection, Weak References

Q51: Explain Python's garbage collection and the `gc` module.

A: Python uses reference counting + cyclic garbage collector. `gc` allows control.

```
import gc
import sys

class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

# Create cycle
a = Node(1); b = Node(2)
a.next = b; b.next = a
del a, b
```

```
gc.collect()  # force collection of
print(gc.get_count())  # (0,0,0)
```

Q52: What is `weakref` and when to use it?

A: `weakref` creates references that don't prevent garbage collection (solves cycles).

```
import weakref

class Cache:
    def __init__(self):
        self._cache = weakref.WeakV

    def set(self, key, value):
        self._cache[key] = value

    def get(self, key):
        return self._cache.get(key)

cache = Cache()
obj = object()
cache.set('key', obj)
```

```
del obj
print(cache.get('key'))  # None, ob
```

Q53: How to detect memory leaks in Python? Use `tracemalloc`.

A:

```
import tracemalloc

tracemalloc.start()
# code that might leak
snapshot = tracemalloc.take_snapsho
top_stats = snapshot.statistics('li
for stat in top_stats[:10]:
    print(stat)
```

Q54: Explain `__slots__` and its memory benefits. {#q54-explain-slots-and-its-memory-benefits }

A: `__slots__` prevents `__dict__` creation, reducing memory per instance.

```
class WithoutSlots:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class WithSlots:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

import sys
wo = WithoutSlots(1,2)
w = WithSlots(1,2)
print(sys.getsizeof(wo.__dict__))
print(sys.getsizeof(w))
```

Q55: Implement a weak reference callback to monitor object deletion.

A:

```
import weakref

def callback(ref):
    print("Object was deleted")

obj = object()
ref = weakref.ref(obj, callback)
print(ref())   # <object at ...>
del obj
print(ref())   # None, and callback
```

8. Advanced Data Structures (collections module)

Q56: Demonstrate `defaultdict`, `Counter`, `OrderedDict`, `deque` with examples.

A:

```
from collections import defaultdict

# defaultdict
dd = defaultdict(list)
dd['key'].append(1)

# Counter
cnt = Counter("abracadabra")
print(cnt.most_common(2))  # [('a',

# OrderedDict (Python 3.7+ dict pre
od = OrderedDict()
```

```
od['a'] = 1; od['b'] = 2
od.move_to_end('a')

# deque
dq = deque([1,2,3])
dq.appendleft(0)
dq.popleft()
```

Q57: Implement a fixed-size queue using `collections.deque`.

A:

```
from collections import deque

class FixedSizeQueue:
    def __init__(self, maxlen):
        self._queue = deque(maxlen=

    def enqueue(self, item):
        self._queue.append(item)

    def dequeue(self):
        return self._queue.popleft(
```

```
    def __len__(self):
        return len(self._queue)
```

Q58: How to use `ChainMap` to merge multiple dictionaries?

A:

```
from collections import ChainMap

defaults = {'theme': 'dark', 'lang'
user = {'lang': 'fr'}
settings = ChainMap(user, defaults)
print(settings['lang'])  # 'fr' (fi
print(settings['theme']) # 'dark'
# ChainMap updates affect first dic
settings['lang'] = 'de'
print(user)  # {'lang': 'de'}
```

Q59: Write a `namedtuple` with default values and

methods.

A:

```
from collections import namedtuple

# Create base
Point = namedtuple('Point', ['x', '
# Default values using __new__ defa
Point.__new__.__defaults__ = (0, 0,
p = Point(1)   # x=1, y=0, z=0
print(p)

# With method - subclass
class PointEx(namedtuple('PointEx',
    __slots__ = ()
    def distance(self):
        return (self.x**2 + self.y*

p2 = PointEx(3,4)
print(p2.distance())
```

Q60: Explain `UserDict`, `UserList`, `UserString`.

A: They are wrapper classes to easily subclass dict/list/str.

```
from collections import UserDict

class CaseInsensitiveDict(UserDict)
    def __setitem__(self, key, valu
        super().__setitem__(key.lov

    def __getitem__(self, key):
        return super().__getitem__(

d = CaseInsensitiveDict()
d['Foo'] = 1
print(d['FOO'])  # 1
```

9. Functional Programming Tools (functools, itertools)

Q61: Implement `functools.partial` manually.

A: Partial freezes some arguments.

```
def partial(func, *args, **kwargs):
    def wrapper(*more_args, **more_
        combined_kwargs = {**kwargs
        return func(*args, *more_ar
    return wrapper

def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
print(square(5))  # 25
```

Q62: What is `itertools.groupby`? Provide example.

A: Groups consecutive items by a key function.

```
from itertools import groupby

data = [("a", 1), ("a", 2), ("b", 3
for key, group in groupby(data, lam
    print(key, list(group))
# a [('a',1),('a',2)]  b [('b',3),(
```

Q63: Implement `functools.reduce` manually.

A:

```
def reduce(function, iterable, init
    it = iter(iterable)
    if initializer is None:
        value = next(it)
    else:
```

```
        value = initializer
    for element in it:
        value = function(value, ele
    return value

print(reduce(lambda a,b: a+b, [1,2,
```

Q64: Write a generator using `itertools.cycle` to create a round-robin scheduler.

A:

```
from itertools import cycle

tasks = ['A', 'B', 'C']
scheduler = cycle(tasks)
for _ in range(10):
    print(next(scheduler))
# A B C A B C ...
```

Q65: How to use `functools.lru_cache` and

its maxsize parameter?

A:

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

# cache info
print(fib.cache_info())  # CacheInf
fib.cache_clear()
```

Q66: Implement `itertools.combinations` manually.

A:

```
def combinations(iterable, r):
    pool = tuple(iterable)
    n = len(pool)
```

```
    if r > n:
        return
    indices = list(range(r))
    yield tuple(pool[i] for i in ir
    while True:
        for i in reversed(range(r))
            if indices[i] != i + n
                break
        else:
            return
        indices[i] += 1
        for j in range(i+1, r):
            indices[j] = indices[j-
        yield tuple(pool[i] for i i

print(list(combinations([1,2,3], 2)
```

Q67: What is `functools.wraps` and why is it needed?

A: It updates wrapper function to look like the original (preserves metadata).

```
from functools import wraps

def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs
    return wrapper
```

Q68: Use `itertools.tee` to implement a sliding window.

A:

```
from itertools import tee, islice

def sliding_window(iterable, n):
    iterators = tee(iterable, n)
    for i, it in enumerate(iterator
        for _ in range(i):
            next(it, None)
    return zip(*iterators)

print(list(sliding_window(range(10)
```

10. Typing and Type Hints

Q69: Explain `TypeVar`, `Generic`, and `Optional`. Write a generic stack class.

A:

```
from typing import TypeVar, Generic

T = TypeVar('T')

class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None
        self._items.append(item)

    def pop(self) -> Optional[T]:
        return self._items.pop() if
```

```
stack = Stack[int]()
stack.push(1)
print(stack.pop())
```

Q70: How to use `@overload` for multiple type signatures?

A:

```
from typing import overload, Union

@overload
def process(data: int) -> int: ...
@overload
def process(data: str) -> str: ...

def process(data: Union[int, str]):
    if isinstance(data, int):
        return data * 2
    else:
        return data.upper()
```

Q71: What is `Protocol` (structural subtyping)? Provide example.

A: `Protocol` defines a set of methods that a class must have.

```
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> None: ...

class Circle:
    def draw(self) -> None:
        print("Circle drawn")

class Square:
    def draw(self) -> None:
        print("Square drawn")

def render(shape: Drawable) -> None
    shape.draw()

render(Circle())  # valid
render(Square())  # valid
```

Q72: Explain `Final`, `Literal`, and `TypedDict`.

A:

```
from typing import Final, Literal,

# Final constant
MAX_SIZE: Final[int] = 100

# Literal exact values
def set_mode(mode: Literal['r', 'w'
    pass

# TypedDict for dictionary structur
class User(TypedDict):
    name: str
    age: int

user: User = {'name': 'Alice', 'age
```

11. Exception Handling and Custom Exceptions

Q73: Write a custom exception hierarchy for a banking app.

A:

```
class BankError(Exception):
    """Base bank exception"""

class InsufficientFundsError(BankEr
    def __init__(self, balance, amo
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot w

class AccountNotFoundError(BankErro
    pass

def withdraw(balance, amount):
    if amount > balance:
```

```
            raise InsufficientFundsErro
        return balance - amount
```

Q74: Explain `else` and `finally` in exception handling.

A: `else` runs if no exception; `finally` always runs.

```
try:
    f = open('file.txt')
except FileNotFoundError:
    print("Not found")
else:
    print("File opened successfully
    f.close()
finally:
    print("Cleanup")
```

Q75: Implement a context manager that suppresses specific exceptions.

A:

```
from contextlib import contextmanag

@contextmanager
def ignore_exception(*exceptions):
    try:
        yield
    except exceptions:
        pass

with ignore_exception(ValueError, Z
    x = int('not a number')  # ignc
```

12. Properties and Slots

Q76: Implement a property with validation using setter.

A:

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Tempe
        self._celsius = value

    @property
    def fahrenheit(self):
```

```
        return self.celsius * 9/5 +

t = Temperature(25)
t.celsius = 30
print(t.fahrenheit)
```

Q77: When to use `__slots__` and what are its limitations? {#q77-when-to-use-slots-and-what-are-its-limitations }

A: Use to reduce memory when creating many instances. Limitations: no `__dict__`, no dynamic attributes, complicates inheritance.

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(1,2)
# p.z = 3  # AttributeError
```

13. Debugging and Profiling

Q78: How to use `pdb` (Python Debugger) effectively? List commands.

A: Set breakpoint with `breakpoint()` (Python 3.7+). Commands: `n` (next), `s` (step), `c` (continue), `p` (print), `l` (list), `q` (quit).

```
def buggy_func(x):
    breakpoint()   # stops here
    return x / 0

buggy_func(5)
```

Q79: Profile a function using `cProfile` and `pstats`.

A:

```
import cProfile
import pstats

def slow():
    sum(range(10_000_000))

profiler = cProfile.Profile()
profiler.enable()
slow()
profiler.disable()
stats = pstats.Stats(profiler)
stats.sort_stats('cumulative').prir
```

Q80: Write a decorator that logs function arguments and return value on exception.

A:

```
import functools
import traceback
```

```
def debug_on_error(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kw
        except Exception as e:
            print(f"Error in {func.
            print(f"Args: {args}, K
            traceback.print_exc()
            raise
    return wrapper
```

14. Testing (unittest, pytest, mock)

Q81: Write a unit test using `unittest.mock` to patch a function.

A:

```
from unittest.mock import patch
import requests

def get_data(url):
    response = requests.get(url)
    return response.json()

# Test
def test_get_data():
    mock_response = {'key': 'value'
    with patch('requests.get') as m
        mock_get.return_value.json.
        result = get_data('http://e
        assert result == mock_respo
```

Q82: Explain `pytest` fixtures with scope.

A: Fixtures manage setup/teardown with scopes (function, class, module, session).

```
import pytest

@pytest.fixture(scope="module")
def db_connection():
    conn = create_connection()
    yield conn
    conn.close()

def test_query(db_connection):
    assert db_connection.query("SEL
```

Q83: How to test asynchronous code with `pytest-asyncio`?

A:

```
import pytest
import asyncio
```

```
@pytest.mark.asyncio
async def test_async_func():
    async def fetch():
        await asyncio.sleep(0.1)
        return 42
    result = await fetch()
    assert result == 42
```

15. Design Patterns in Python

Q84: Implement the Observer pattern.

A:

```
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(obse

    def notify(self, data):
        for obs in self._observers:
            obs.update(data)

class Observer:
    def update(self, data):
        print(f"Received: {data}")

subject = Subject()
```

```
obs = Observer()
subject.attach(obs)
subject.notify("Hello")
```

Q85: Implement the Factory pattern with a registry.

A:

```
class Animal:
    def speak(self): pass

class Dog(Animal):
    def speak(self): return "Woof"

class Cat(Animal):
    def speak(self): return "Meow"

class AnimalFactory:
    _registry = {}

    @classmethod
    def register(cls, name, animal_
        cls._registry[name] = anima
```

```
    @classmethod
    def create(cls, name):
        animal_cls = cls._registry.
        if not animal_cls:
            raise ValueError(f"Unkr
        return animal_cls()

AnimalFactory.register('dog', Dog)
AnimalFactory.register('cat', Cat)

animal = AnimalFactory.create('dog'
print(animal.speak())
```

Q86: Explain the Context Manager pattern (already done). Show a custom implementation of `with` statement.

A: (Covered earlier) but add a `__enter__` / `__exit__` example.

Q87: Implement the Strategy pattern using functions.

A:

```
def bubble_sort(arr):
    # ... sort
    return arr

def quick_sort(arr):
    # ... sort
    return arr

class Sorter:
    def __init__(self, strategy):
        self.strategy = strategy

    def sort(self, data):
        return self.strategy(data)

sorter = Sorter(bubble_sort)
print(sorter.sort([3,1,2]))
```

16. Working with C Extensions / ctypes / Cython

Q88: How to call a C function from Python using `ctypes`?

A: Load a shared library and call.

```
import ctypes

libc = ctypes.CDLL("libc.so.6")  #
libc.printf(b"Hello %s\n", b"World'

# Custom library
lib = ctypes.CDLL("./mylib.so")
lib.my_function.argtypes = (ctypes.
lib.my_function.restype = ctypes.c_
```

Q89: What is Cython and how does it speed up

code?

A: Cython compiles Python-like code to C, yielding C extensions.

```
# example.pyx
def fib(int n):
    cdef int a = 0, b = 1, i
    for i in range(n):
        a, b = b, a+b
    return a
```

17. Advanced File I/O and Serialization

Q90: Compare `pickle`, `json`, and `msgpack`. When to use each?

A: JSON for interoperability; pickle for Python-only with objects; msgpack for binary compact.

```
import pickle, json, msgpack

data = {'a': 1, 'b': [1,2,3]}

# Pickle
with open('data.pkl', 'wb') as f:
    pickle.dump(data, f)

# JSON
with open('data.json', 'w') as f:
    json.dump(data, f)

# msgpack
```

```
with open('data.msgpack', 'wb') as
    msgpack.dump(data, f)
```

Q91: Implement a custom `Encoder` / `Decoder` for JSON to handle `datetime`.

A:

```
import json
from datetime import datetime

class DateTimeEncoder(json.JSONEnco
    def default(self, obj):
        if isinstance(obj, datetime
            return obj.isoformat()
        return super().default(obj)

def datetime_decoder(dct):
    for key, value in dct.items():
        if isinstance(value, str):
            try:
                dct[key] = datetime
            except ValueError:
                pass
```

```
    return dct

data = {'now': datetime.now()}
json_str = json.dumps(data, cls=Dat
decoded = json.loads(json_str, obje
```

18. Context Variables, Variable Scope, Closures

Q92: Explain `contextvars` module and its use in asyncio.

A: `ContextVar` provides isolated per-task/thread context, useful for tracing.

```
from contextvars import ContextVar
import asyncio

request_id_var: ContextVar[str] = C

async def handle():
    req_id = request_id_var.get()
    print(f"Processing {req_id}")

async def middleware(req_id):
    token = request_id_var.set(req_
    try:
```

```
        await handle()
    finally:
        request_id_var.reset(token)

asyncio.run(middleware("ABC123"))
```

Q93: Write a closure that maintains state without classes.

A:

```
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

c1 = make_counter()
print(c1(), c1())  # 1 2
```

19. AST, Bytecode, Dynamic Code Execution

Q94: Use `ast` module to parse and modify code.

A: Detect all function names.

```
import ast

code = """
def foo(): pass
def bar(): pass
"""

tree = ast.parse(code)
func_names = [node.name for node in
print(func_names)  # ['foo', 'bar']
```

Q95: What is the difference between `eval`, `exec`, and

`compile`?

A: `eval` evaluates an expression; `exec` executes statements; `compile` compiles to code object.

```
x = 5
result = eval("x * 2")       # retur
exec("y = x + 3")            # modif
code = compile("print('hi')", '<str
exec(code)                   # hi
```

20. Miscellaneous (GIL, Signal Handling, etc.)

Q96: How to handle UNIX signals in Python?

A:

```
import signal, time

def handler(signum, frame):
    print("SIGINT received")

signal.signal(signal.SIGINT, handle
print("Press Ctrl+C")
time.sleep(10)
```

Q97: Implement a simple event loop using `selectors`.

A:

```
import selectors
import socket

sel = selectors.DefaultSelector()
sock = socket.socket()
sock.setblocking(False)
sock.bind(('localhost', 8080))
sock.listen()
sel.register(sock, selectors.EVENT_

def accept(sock):
    conn, addr = sock.accept()
    print(f"Accepted {addr}")

while True:
    events = sel.select()
    for key, mask in events:
        accept(key.fileobj)
```

Q98: Explain Python's `atexit` module.

A: Register functions to run at interpreter shutdown.

```python
import atexit

def cleanup():
    print("Cleaning up")

atexit.register(cleanup)
```

Q99: What is `sys.setrecursionlimit` and its dangers?

A: Increases maximum recursion depth, can cause segmentation fault if too high.

```python
import sys
sys.setrecursionlimit(10000)

def deep(n):
    if n == 0:
        return
    deep(n-1)
deep(5000)  # works, but risk of st
```

Q100: Write a function that retries an operation with exponential backoff and jitter.

A:

```
import random, time

def retry_with_jitter(func, max_ret
    for attempt in range(max_retrie
        try:
            return func()
        except Exception as e:
            delay = min(base_delay
            print(f"Attempt {attemp
            time.sleep(delay)
    raise Exception("Max retries ex

def unstable():
    if random.random() < 0.8:
        raise ValueError("Fail")
    return "Success"

print(retry_with_jitter(unstable))
```